

IT 2012 - Unstructured Data

Lab 9 - Data Cleaning

Assist. Prof. Dr. Dželila Mehanović

Introduction

In Lab 8 you built the data exploration layer. You loaded data from MongoDB into pandas DataFrames, inspected it with EDA tools, and wrote a quality report that listed all the problems found in the data. In Lab 9 the focus will be to fix those problems.

Data cleaning is the process of correcting or removing wrong, incomplete, or inconsistent values so that the data is trustworthy before it is used for analysis or modelling. A clean dataset is not perfect, but every decision you make while cleaning it must be documented so that another person can reproduce your work.

Note: This lab continues directly from Lab 8. Make sure your MongoDB collections from previous labs are populated before you start. If you need to repopulate them, run `run_pipeline.py` from Lab 8 first.

Learning Objectives

By the end of this lab you will be able to:

- Analyse missing value patterns and decide whether to drop, fill, or impute them
- Handle missing values correctly using `dropna`, `fillna`, and `interpolate`
- Clean string columns by removing whitespace, normalising case, and fixing format
- Use regular expressions to clean complex text patterns
- Remove duplicate rows using exact and subset-based matching
- Convert columns to the correct data type using `astype`, `to_datetime`, and `to_numeric`
- Validate cleaned data with assert statements and schema checks
- Run the test suite and interpret results
- Document every cleaning decision with a clear rationale
- Build a reusable cleaning module that slots into the existing pipeline
- Log all cleaning activities to `pipeline.log`

This lab introduces a new **src/cleaning/** package dedicated to data preprocessing tasks, along with a new **tests/** folder for validating cleaning functionality and a new **processed/cleaned/** directory for storing generated cleaned datasets.

In **src/cleaning/**, add **__init__.py** to initialize the package, **missing_handler.py** to detect and handle missing values, **string_cleaner.py** to standardize and normalize text columns, **deduplicator.py** to identify and remove duplicate records, **type_converter.py** to convert columns into appropriate data types, **validator.py** to verify data consistency and integrity, and **clean_pipeline.py** to combine all cleaning steps into one reusable workflow.

In **tests/**, add `__init__.py` and `test_cleaning.py` to implement unit tests for the cleaning modules. In **processed/cleaned/**, generate `clean.csv` as the final cleaned dataset produced by the pipeline. Additionally, create **notebooks/lab9_data_cleaning.ipynb** as the submission notebook demonstrating the full data cleaning process, while updating `run_pipeline.py` and `requirements.txt` to support the new modules and dependencies.

Part 1 - Install Required Libraries and Tools

Update `requirements.txt`

Open `requirements.txt` and add these lines if they are not already there:

```
# requirements.txt
pytest>=7.4.0
pytest-cov>=4.1.0
```

Then activate your virtual environment in the VS Code terminal and install:

```
pip install -r requirements.txt
```

Part 2 - Handling Missing Values

Missing values are one of the most common problems in real-world data. Before you can fix them you need to understand why they are missing. There are three patterns:

Pattern	What it means and what to do
MCAR (Missing Completely At Random)	The missing value has no relationship to any other variable. Safe to drop or fill with a simple statistic like mean or median.
MAR (Missing At Random)	The missing value depends on another column that you can observe. Fill using values from the related column.
MNAR (Missing Not At Random)	The value is missing because of the value itself. For example, very low-rated movies might have no revenue recorded. Simple drop or fill will introduce bias.

Create the file [src/cleaning/missing_handler.py](#) and implement functions for handling missing data in your project dataset cleaning pipeline. The code should generate a missing-value report, remove rows with missing critical identifiers, fill descriptive text fields with placeholders, replace unrealistic zero values with NaN, fill numeric missing values with medians, and drop columns whose missing-data ratio is too high.

In [notebook/lab9_data_cleaning.ipynb](#), add separate notebook cells to set up imports and logging, load the raw CSV file produced in Lab 8, and inspect missing values using the `report_missing` function from `src/cleaning/missing_handler.py`. The notebook should also display the missing-value summary and save it as `processed/cleaned/missing_report.csv` for later use in the cleaning pipeline.

Part 3 - String Cleaning

String columns in data are often inconsistent. Titles might have extra spaces, genres might be in different cases, or language codes might be stored with mixed capitalisation. The pandas `.str` accessor lets you apply string methods to every row at once without a Python loop.

Create the file `src/cleaning/string_cleaner.py` and implement functions for cleaning text-based columns in your dataset using vectorized pandas string operations instead of Python loops. Examples may include titles, names, categories, descriptions, codes, or labels depending on your project.

In [notebook/lab9_data_cleaning.ipynb](#) add a new notebook cell to apply the string-cleaning functions from [src/cleaning/string_cleaner.py](#) on a copy of the raw dataset so the original data stays unchanged. This cell should clean titles, normalize language codes, clean overview text, extract the release year into a new column, and then print a preview of the updated columns to confirm the changes.

Part 4 - Regular Expressions for Complex Cleaning

Regular expressions let you match patterns rather than exact strings. This is useful when you need to find values that follow a shape, for example a date, an ID number, or a word that might appear in different forms.

NOTE: Always write regex patterns as raw strings in Python: `r'd{4}'` not `'d{4}'`. Raw strings tell Python not to process backslashes, which the regex engine needs.

Open the existing [src/analytics/regex_ops.py](#) file you created in Lab 8 and extend it by adding new regex-based helper functions for data cleaning and validation at the bottom of the file. These functions should detect invalid date formats and language codes, extract numeric values from text strings, and flag overviews that are too short, helping identify suspicious or inconsistent records before further analysis.

Part 5 - Removing Duplicates

Duplicate rows appear when the same data is collected from multiple API calls, when pagination overlaps, or when data from different sources is combined. There are two kinds of duplicates you need to handle:

- **Exact duplicates:** every column in two rows is identical
- **Key duplicates:** only the ID column (tmdb_id or id) is duplicated, but other fields might differ slightly because they were collected at different times

Note: Always normalise string columns before deduplicating. For example 'The Matrix' and ' The Matrix' (with a leading space) would not be detected as duplicates without cleaning first.

Create the file [src/cleaning/deduplicator.py](#) and implement functions for detecting and removing duplicate records in the dataset from the dataset in a clear and testable way. The code should remove fully identical rows, drop repeated record IDs or primary keys, remove duplicate titles with the same release date, and provide a helper function to count duplicates in a chosen column before or after cleaning. In **lab9_data_cleaning.ipynb**, add a new cell to import the deduplication functions from **src/cleaning/deduplicator.py** and apply them to the dataset step by step. This cell should display the number of rows before cleaning, remove fully identical records, count duplicate identifiers before cleaning, drop repeated IDs, and print the updated row totals after each stage to verify the cleaning results.

Part 6 - Data Type Conversion

When pandas reads data from a CSV or MongoDB export it often assigns every column the object (string) type because it cannot safely infer types. You need to convert each column to the type it actually represents so that arithmetic and comparisons work correctly.

Function	Use case
<code>astype('int64')</code>	Convert a column that should be whole numbers
<code>astype('float32')</code>	Convert a column that should be decimal numbers, uses less memory than float64
<code>pd.to_datetime()</code>	Convert a string date column to a proper datetime type
<code>pd.to_numeric(errors='coerce')</code>	Convert a string column to numbers, setting invalid values to NaN
<code>astype('category')</code>	Convert a low-cardinality text column to save memory

<code>astype('Int64')</code>	Nullable integer type that allows NaN (unlike plain <code>int64</code>)
------------------------------	--

Create the file [src/cleaning/type_converter.py](#) and implement functions that safely convert dataset columns into their correct data types while handling invalid values with `errors='coerce'`. The code should convert release dates to `datetime`, transform numeric columns into appropriate float and integer types, convert low-cardinality text columns into categorical data, and include a memory report function to compare memory usage before and after conversion.

In `lab9_data_cleaning.ipynb`, add a new notebook cell to import the type conversion functions from `src/cleaning/type_converter.py` and apply them to the cleaned dataset. This cell should keep a copy of the dataset before conversion for comparison, run the date, numeric, and category conversions, print the updated data types, and display a memory usage report to show the effect of the conversions.

[Part 7 - Data Validation](#)

Validation checks that the cleaned data actually meets your expectations. A value can be non-null and correctly typed but still be wrong, for example a `vote_average` of 15 when TMDb uses a 0-10 scale, or a movie with a release year of 1800 before cinema was invented.

Use Python's built-in `assert` statement. If the condition is `False`, Python raises an `AssertionError` immediately and tells you what failed. This is called failing fast.

[Part 8 - The Reusable Cleaning Pipeline](#)

Now that all cleaning functions exist in separate modules, you will combine them into one clean pipeline function. This function takes a raw `DataFrame` and returns a clean one by applying every step in the correct order. A cleaning pipeline should be a single function that can be called from `run_pipeline.py` or from a notebook cell. It must log every step so you can trace what happened to the data.

Create the file [src/cleaning/clean_pipeline.py](#) and implement the main cleaning pipeline that combines all previously created cleaning modules into one reusable workflow for the project dataset. The code should copy the raw `DataFrame`, run each cleaning step in the correct order, handle missing values, duplicates, string cleaning, year extraction, type conversion, and validation, and finally save the cleaned dataset as `processed/cleaned/cleaned_data.csv`.

Part 9 – Writing and Running Tests with pytest

Every cleaning function you wrote is a piece of code. Code needs tests. A test is a small function that checks whether your cleaning function behaves correctly on a specific input. pytest discovers tests automatically. Any Python file whose name starts with `test_` and any function inside it whose name starts with `test_` will be picked up and run.

pytest command	What it does
<code>pytest</code>	Run all tests in the project
<code>pytest tests/test_cleaning.py</code>	Run only the cleaning tests
<code>pytest -v</code>	Show each test name with PASSED or FAILED
<code>pytest -v --tb=short</code>	Show short traceback on failure
<code>pytest -k test_title</code>	Run only tests whose name contains 'test_title'
<code>pytest --cov=src/cleaning</code>	Measure how much of the cleaning code is tested

Create the file [tests/test_cleaning.py](#) and write a set of unit tests to verify that the main cleaning functions behave correctly on small sample project data with intentional errors. The test file should include shared fixture data and multiple test functions for missing-value handling, string cleaning, deduplication, type conversion, and validation, so that each important cleaning step can be checked independently.

After that, run `pytest tests/test_cleaning.py -v` from the project root in the VS Code terminal to execute all tests and confirm that the cleaning pipeline works as expected. If any test fails, carefully read the assertion error, fix the related function, and run the tests again until all tests pass. In terminal you should see something like this:

collected 17 items

```
src/tests/test_cleaning.py::test_drop_rows_missing_title_removes_null PASSED
src/tests/test_cleaning.py::test_drop_rows_missing_title_removes_empty_string PASSED
src/tests/test_cleaning.py::test_fill_missing_overview_no_nulls_remain PASSED
src/tests/test_cleaning.py::test_fill_missing_overview_uses_placeholder PASSED
src/tests/test_cleaning.py::test_replace_zero_with_nan_on_budget PASSED
src/tests/test_cleaning.py::test_clean_title_strips_whitespace PASSED
src/tests/test_cleaning.py::test_clean_language_code_lowercases PASSED
src/tests/test_cleaning.py::test_extract_year_creates_column PASSED
src/tests/test_cleaning.py::test_extract_year_correct_values PASSED
src/tests/test_cleaning.py::test_drop_exact_duplicates_removes_copies PASSED
src/tests/test_cleaning.py::test_drop_duplicate_ids_keeps_first PASSED
src/tests/test_cleaning.py::test_count_duplicates_returns_correct_number PASSED
src/tests/test_cleaning.py::test_convert_dates_produces_datetime_type PASSED
src/tests/test_cleaning.py::test_convert_dates_bad_values_become_nat PASSED
src/tests/test_cleaning.py::test_validate_no_null_titles_passes_on_clean_data PASSED
src/tests/test_cleaning.py::test_validate_no_null_titles_fails_on_null PASSED
```

If any test fails, read the error message carefully. It will tell you which assertion failed and what values it saw. Fix the corresponding function and run the tests again.

Part 10 – Integrate Cleaning into run_pipeline.py

Open the existing run_pipeline.py file and add a new cleaning step that automatically runs after the analytics pipeline from last week has finished. This code should load the raw CSV file generated in Lab 8, pass it through run_cleaning_pipeline() from **src/cleaning/clean_pipeline.py**, save the cleaned dataset to **processed/cleaned/**, and log the progress so the full pipeline can perform both analytics and cleaning in one execution.

Assignment Tasks:

1. Missing value analysis (10%) – Identify, visualise, and understand missing data patterns in the dataset.
2. Missing value handling (10%) – Apply different strategies such as dropping rows, filling with constants, or using median values to resolve missing data.
3. String cleaning (10%) – Clean text columns by fixing whitespace, letter casing, and formatting inconsistencies.
4. Regex cleaning (10%) – Use regular expressions to detect, validate, and clean complex text patterns in selected columns.
5. Deduplication (10%) – Remove exact duplicate rows and repeated IDs while reporting row counts before and after cleaning.
6. Type conversion (10%) – Convert columns into the correct data types such as dates, numeric values, and categorical variables.
7. Validation (10%) – Perform logical checks and assertions to confirm that the cleaned dataset is accurate and ready for analysis.
8. Pytest tests (15%) – Create multiple unit tests for the cleaning functions and ensure all tests pass successfully.
9. Pipeline (10%) – Implement the full cleaning workflow inside a reusable pipeline module that can be executed with a single function call.

NOTE: Apply all tasks to your own project dataset. Column names and validation rules must reflect your domain. Movies examples shown earlier are illustrative only.